

**Documentation
for
EasyField Pro – Map Creator & Pathfinding
(Zenject Edition)**

Table of contents.

Introduction.....	3
Package architecture.	4
Demonstration Scenes Overview	5
Code Overview.	6
Cell indexes for grids.....	9
Questions and support.....	12

Introduction.

This is the official documentation file for the asset «**EasyField Pro – Map Creator & Pathfinding (Zenject Edition)**», available in Unity Asset Store:

<https://u3d.as/49nf>

This framework is engineered to serve as an architectural foundation for 2D top-down games. It delivers a decoupled core framework, a highly optimized universal pathfinding engine, and a set of pre-configured demonstration scenes designed to serve as a robust starting point for your next project.

EasyField Pro is built from the ground up according to **Clean Architecture** and **Data-View Separation** patterns. Within this ecosystem, data models and view layers are entirely decoupled: data models handle only pathfinding mathematics and graph states (without any knowledge of rendering or sprites), while the view layers focus exclusively on visual presentation. Instead of maintaining direct object references, Data and View components are bound dynamically via shared unique identifiers (IDs). High-speed internal storages leverage these IDs to provide instant $O(1)$ lookup times for any entity on the field.

Furthermore, the framework completely avoids the traditional, heavy approach of binding events to every separate entity. Instead, when an entity changes its parameters, the action is processed by a centralized component. This controller fires a single global event, passing only the unique ID of the modified entity and its new state data. All scene systems and listeners bind directly to these centralized components rather than individual objects, drastically reducing event overhead and preventing memory leaks.

This specific edition of the framework is built entirely on top of the popular dependency injection plugin: **Zenject/Extenject**. *Please ensure to install it together with EasyField, and consider building your entire game project around this DI architecture.* The plugin is available for free in Unity Asset Store:

<https://assetstore.unity.com/packages/tools/utilities/extenject-dependency-injection-ioc-157735>

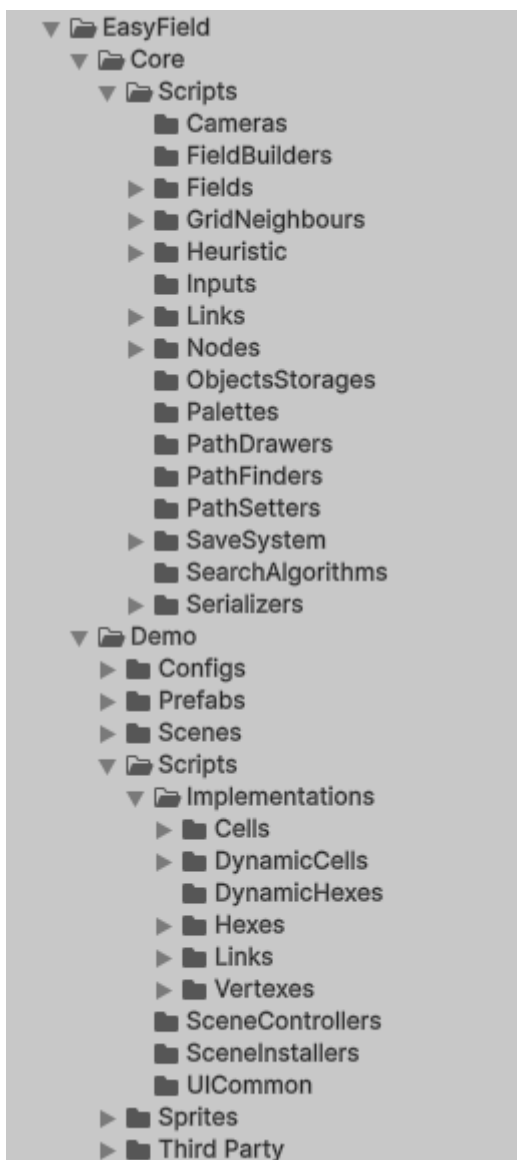
As a first step, it is highly recommended to open and run the provided **demonstration scenes** to see the framework in action. Once you choose the field configuration that most closely matches your game mechanics, you can take its specific installers and controllers as your project baseline and start learning the code.

Package architecture.

The package is separated into two main sections, each isolated with its own assembly definitions: **Core** and **Demo**.

The **Core** section contains the foundational framework infrastructure, core pathfinding algorithms, generic abstractions, and basic data models for entities. This is the primary part of the framework that you will integrate into your production environment. It is completely independent of the visual styling or resources used in the samples.

The **Demo** section contains the 12 pre-configured demonstration scenes, which showcase concrete implementations of the basic entities, such as grid cells, hexagonal tiles, and freeform graph vertices. It also includes their auxiliary UI modules, controllers, and related graphics resources. Since the entire architecture is loosely coupled, you can safely remove the Demo folder or its individual parts before building your final game, carrying over only the specific implementations and layouts you need as a baseline.



Demonstration Scenes Overview

EasyField Pro comes packed with 12 carefully pre-configured demonstration scenes, each complete with unique installers and scene controllers designed to showcase various field architectures. Additionally, the package includes 12 sample map save files (one tailored for each scene). To ensure layout compatibility, please load the save file that matches the current active scene name. These data files are located within the «Demo/Scenes/Maps» directory.

- **Scenes 1a, 1b (Square Grids):** Demonstrate standard rectangular grid systems with square cells. These scenes showcase 4-directional and 8-directional movement (featuring diagonal cost scaling). Path cost calculation strictly depends on underlying cell terrain types.
- **Scenes 2a, 2b, 2c (Spatial Graphs):** Showcase continuous, unaligned node networks where vertices can be freely dragged in 2D space within field boundaries. Links between nodes are created manually at runtime. These samples also demonstrate live edge cost editing and strict blocking mechanics for specific nodes or links.
- **Scenes 3a, 3b, 3c, 3d (Hexagonal Grids):** Demonstrate all four geometric variations of hexagonal coordination – pointy-topped and flat-topped orientations across odd and even staggering layout patterns. Path cost calculation strictly depends on underlying cell terrain types.
- **Scenes 4a, 4b (Dynamic Square Grids):** Feature the same rectangular grid infrastructure as in Scenes 1*, but here cells can be dynamically deleted or re-spawned. They also allow the creation of custom, unaligned links that function as teleports or portals.
- **Scene 5a (Dynamic Hexagonal Grids):** Feature the same hexagonal grid infrastructure as in Scenes 3*, but here cells can be dynamically deleted or re-spawned. It also allows the creation of custom, unaligned links that function as teleports or portals.

Each of these scenes in Unity is driven by two main components that coordinate operations at startup: a **SceneInstaller** and a **SceneController** (with class names reflecting their corresponding scene numbers).

The **SceneInstaller** is a dedicated Zenject component responsible for defining the dependency injection container rules and binding abstract interfaces to concrete system implementations. Placed directly onto the scene's `SceneContext` `GameObject`, it automatically executes its bindings before any game logic starts.

The **SceneController** acts as a centralized mediator pattern, binding decoupled scene components (such as managers, UI systems, and click handlers) to each other through an ID-driven event pipeline. It automatically hooks into the application lifecycle at scene startup by implementing Zenject's `IInitializable` interface.

Code Overview.

All code in EasyField Pro is 100% open, so you can easily write your own custom implementations for any of the interfaces.

The pathfinding system is built around two core entities: **Nodes** and **Links**. Following the Data-View Separation pattern, both are split into data models and views. They do not know about each other directly and are connected using a shared unique ID. You can choose any generic type for this ID (int, string, Guid, Vector2Int, etc). While nodes use a single ID, links use a dual composite ID that consists of two node IDs (the start node and the finish node).

Nodes

Nodes represent points on the field. A node's position can be stored either as world coordinates or as grid indexes (depending on the field type). Nodes also have a «blocked» property (on/off) that allows you to temporarily exclude them from pathfinding. In the demo scenes, nodes are implemented as grid Cells, Hexes, and freeform spatial Vertexes.

Links

Links are set between two nodes and represent movement connections. In many games, links are invisible and do not use a view component. In tile-based games, links are usually generated at runtime based on the terrain cost of the cells. Additional links can be created to work as teleports, portals, or shortcuts. In freeform graphs, links are managed manually. Each link has its own movement cost property and a blocked state.

Fields

Fields are visual components that contain node views and link views. They automatically control graphic resizing, bounds, and collider dimensions. Fields are available as freeform **Spatial Fields** (where nodes can be dragged to any coordinate) and **Grid Fields** (which come in square and hexagonal layouts).

FieldBuilders

These high-level components manage the creation of fields. They can generate a new empty field of a specified size or construct it from a saved json file. FieldBuilders command the EntityCreator components to manage the lifecycle of nodes and links. Examples include, according to specified scenes, such classes as CellsFieldBuilder, HexFieldBuilder, DynamicCellsFieldBuilder, DynamicHexFieldBuilder and VertexesFieldBuilder.

EntityCreators

High-level managers that handle the creation and deletion of basic field entities (nodes and links). They spawn both the data model and the view at the same time, using the same shared ID. They connect low-level factories with live object storages. Examples include, according to specified scenes, such classes as CellsNodesCreator, VertexesNodesCreator, LinksCreator and VertexesLinksCreator.

Factories and pools

Factories in EasyField Pro work as simple wrappers around object pools. They handle the low-level spawning and despawning of specific node and link data models and view prefabs. Each factory and pool correspond to a single specific object type (data model or view for specific node/link implementation).

Object storages

Simple collection wrappers built around dictionaries (`DictTypeStorage`) and two-dimensional arrays (`GridTypeStorage`). They provide quick $O(1)$ access to basic field entities (data models and views) via their unique IDs.

LinksProviders

These classes let you quickly find incoming or outgoing connections for a specific node. Implemented as three classes: while `StoredLinksProvider` acts as an advanced storage wrapper for manually created links, `GridDynamicLinksProvider` instead creates links at runtime, designed to be used for tile-based fields. `CombinedLinksProvider` merges both approaches together, which is specifically used for tile-based fields that feature custom teleports and portals.

Scene manipulators & Input Handlers

Simple components used to control interactions in the scene (clicking, dragging, selection, property changers). To optimize performance and prevent memory leaks, these handlers pass only pure IDs through a centralized event system instead of referencing heavy object instances directly.

Serializers

The framework supports serialization into both **JsonUtility** and **Newtonsoft.Json** formats. You can save your maps as raw text JSON files or convert them into byte arrays. Text-based JSON files are highly comfortable for human reading and are easily editable by hand. For production or large maps, you can use the built-in GZip compression decorator to seamlessly compress your binary files and drastically reduce their size on disk.

Save system

A flexible data saving architecture that handles map persistence through several components:

- **DTOs (Data Transfer Objects):** Simple serializable classes used to store only the parameters needed for a save file.
- **Mappers:** Simple classes used to convert runtime data models into DTOs.
- **FileDtoGateways:** The bridge between a file on the disk and a main save DTO.
- **FilePathProviders:** Simple path managers that support constant hardcoded paths or native standalone file dialogs.
- **FieldSaveDtoProviders:** Components that collect data from all object storages and compile them into a main DTO save file.

SearchAlgorithms

The core pathfinding system that uses a classic generic A* algorithm. If the injected heuristic provider returns a zero cost, the algorithm automatically turns into Dijkstra's algorithm, checking all possible paths.

GridNeighboursProviders

Services for two-dimensional arrays that find adjacent nodes. They provide 4-way or 8-way adjacent cell index lookups for rectangular grids, and 6-way lookups for hexagonal staggering patterns.

Heuristics & Cost Providers

Heuristic providers calculate the estimated distance to the destination (supporting Manhattan, Chebyshev, Octile, Euclidean, and Hexagonal functions). Cost providers calculate the actual movement cost, which is useful for tile-based games where cost depends on the terrain type (using the initial cell, the target cell, or an average of both).

PathDrawers

Components used to visualize the calculated path on the screen. The framework includes a LineRenderer-based path drawer and a cell-marker approach that highlights individual node view components.

PathFinders

Higher-level pathfinding utilities available in two variations: PathFinder (handles direct calculation) and CachePathFinder (caches previously calculated paths to optimize performance on repetitive queries, requiring a manual clear when the field layout changes).

PathSetters

Centralized state components that track the active start and finish landmarks of a pathfinding request, triggering scene updates whenever a target node changes.

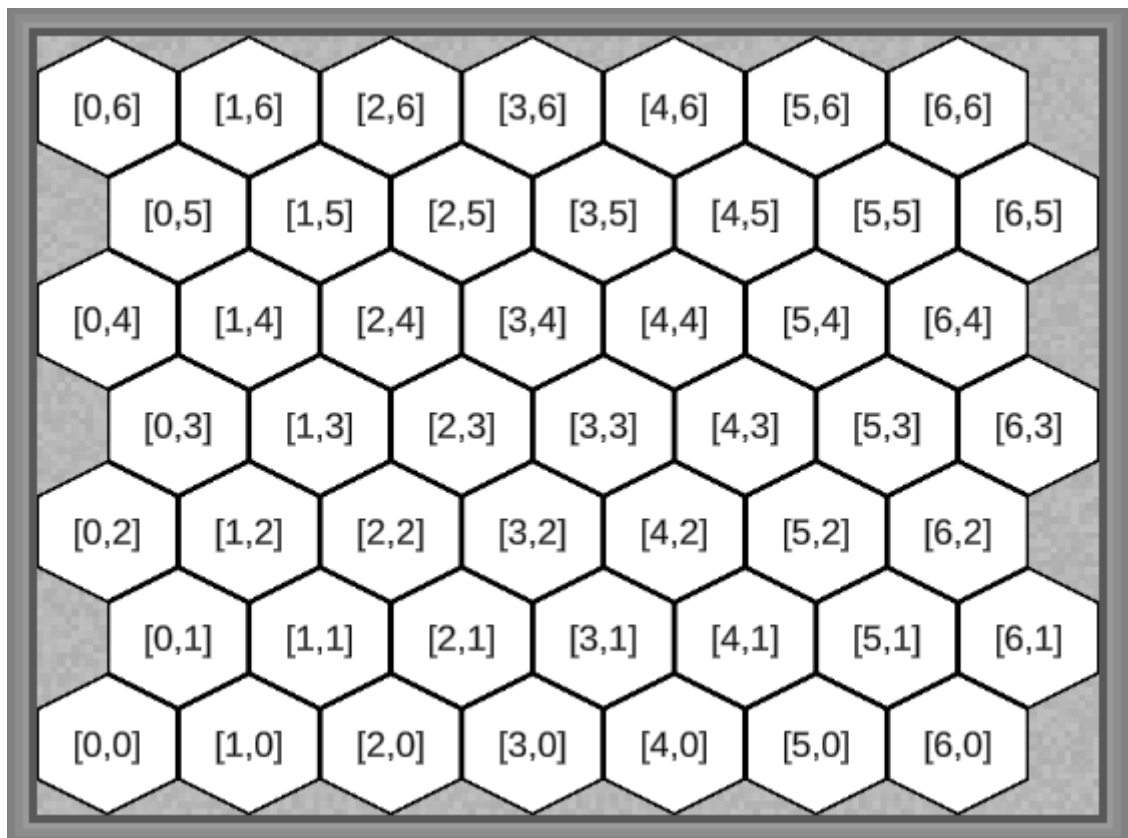
Cell indexes for grids.

This is how cell indexes are distributed for grid-based fields.

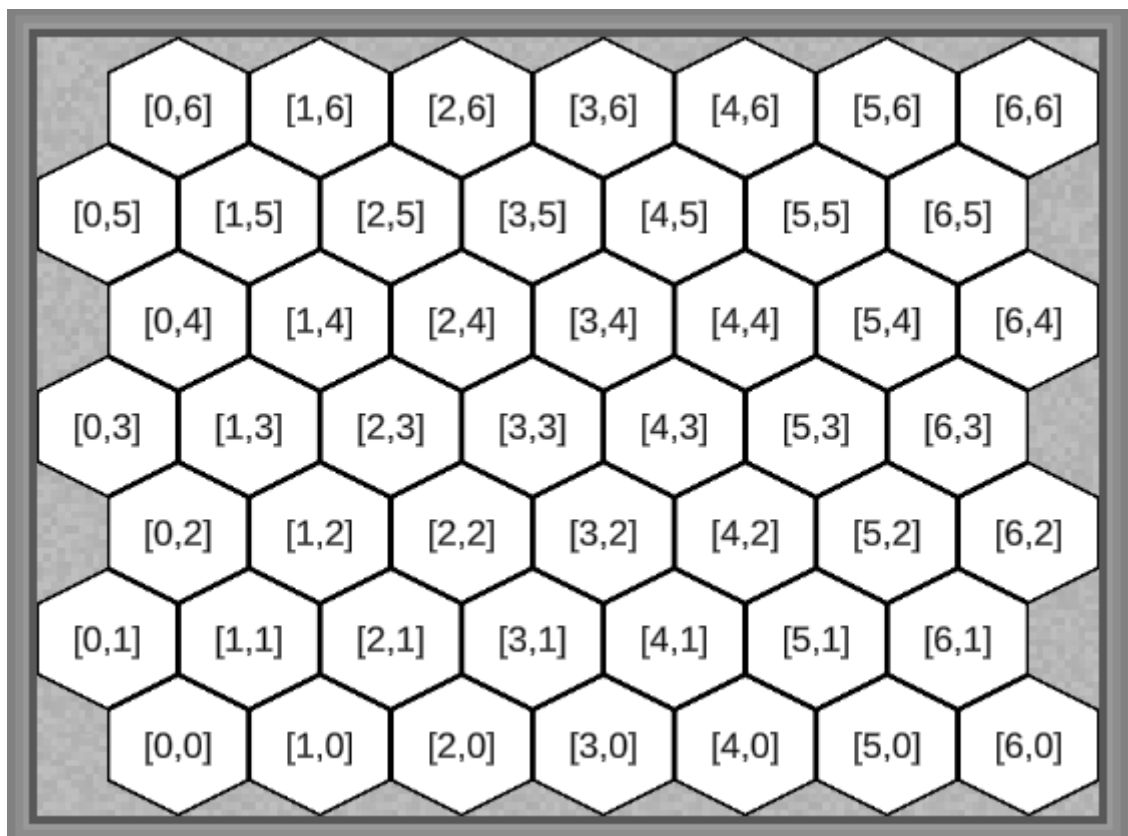
- Square grid field:

[0,6]	[1,6]	[2,6]	[3,6]	[4,6]	[5,6]	[6,6]
[0,5]	[1,5]	[2,5]	[3,5]	[4,5]	[5,5]	[6,5]
[0,4]	[1,4]	[2,4]	[3,4]	[4,4]	[5,4]	[6,4]
[0,3]	[1,3]	[2,3]	[3,3]	[4,3]	[5,3]	[6,3]
[0,2]	[1,2]	[2,2]	[3,2]	[4,2]	[5,2]	[6,2]
[0,1]	[1,1]	[2,1]	[3,1]	[4,1]	[5,1]	[6,1]
[0,0]	[1,0]	[2,0]	[3,0]	[4,0]	[5,0]	[6,0]

- OddR hexagonal grid field:



- EvenR hexagonal grid field:



-

-
- A 7x7 hexagonal grid representing a 2D lattice. Each hexagon contains a pair of coordinates (i, j) in black text. The coordinates range from $(0,0)$ at the bottom-left to $(6,6)$ at the top-right. The grid is enclosed in a thick black border.

Questions and support.

We are committed to helping you integrate EasyField Pro into your project seamlessly. If you run into any unexpected bugs, architecture questions, or need general assistance, please do not hesitate to reach out. You can contact us directly via support email at: Omish1990@proton.me